

HITBOXKING

Plan de Trabajo — Proyecto Semestral
Arquitectura de Microservicios con Spring Boot

13
Microservicios

5
Epicas

26
Tareas

Java 21
Stack

CRITICA	Bloquea todo lo demas	ALTA	Necesaria para el flujo	MEDIA	Importante pero no urgente	BAJA	Complementari
----------------	-----------------------	-------------	-------------------------	--------------	----------------------------	-------------	---------------

Full Stack I — Ingenieria de Software | Duoc UC

TABLA DE PUERTOS

Referencia rapida de todos los microservicios y sus puertos asignados.

Microservicio	Puerto
config-server	8888
eureka-server	8761
api-gateway	8080
ms-auth	8081
ms-weapons	8082
ms-agents	8083
ms-maps	8084
ms-persistence	8085
ms-training-engine	8086
ms-stats-maps	8087
ms-stats-weapons	8088
ms-stats-agents	8089
ms-analytics-core	8090

Infraestructura Base

Levanta los servicios fundacionales sin los cuales ningun microservicio puede arrancar. Estas tareas deben completarse primero en el orden indicado.

HIT-1

CRITICA

Crear y configurar config-server

DESCRIPCION

El primer servicio que debe levantarse. Todos los microservicios le consultan sus propiedades al arrancar via `spring.cloud.config.client`. Sin el, ningun otro servicio puede iniciar.

PASOS

- Crear proyecto Maven en `start.spring.io`: Java 21, Spring Boot 3.2.x, Jar, dependencia Config Server unicamente
- Anotar la clase main con `@EnableConfigServer`
- Crear carpeta `/config-repo` con `application.yml` y un `[nombre-servicio].yml` por cada microservicio
- Configurar `application.yml`: puerto 8888, `spring.cloud.config.server.native.searchLocations=classpath:/config-repo`
- Agregar `spring.cloud.config.fail-fast=true` en los clientes
- Package base: `cl.nexusarena.configserver`

CRITERIOS DE ACEPTACION

- ✓ El servicio arranca en el puerto 8888 sin errores en consola
- ✓ GET `http://localhost:8888/ms-auth/default` retorna JSON con propiedades configuradas
- ✓ GET `http://localhost:8888/ms-weapons/default` retorna propiedades del servicio weapons
- ✓ Al modificar un `.yml` en `config-repo` el cambio se sirve inmediatamente
- ✓ El servicio aparece como UP en logs sin stack traces

Crear y levantar eureka-server

DESCRIPCION

Registro central de instancias. Todos los microservicios se registran aqui al arrancar y se descubren entre si por nombre logico sin IPs hardcodeadas. Debe levantarse inmediatamente despues del config-server.

PASOS

- Crear proyecto Maven: Java 21, Spring Boot 3.2.x, Jar, dependencia Eureka Server
- Anotar la clase main con `@EnableEurekaServer`
- `application.yml`: puerto 8761, `register-with-eureka`: false, `fetch-registry`: false
- Package base: `cl.nexusarena.eurekaserver`
- Documentar la URL del dashboard en el README del repositorio

CRITERIOS DE ACEPTACION

- ✓ Dashboard de Eureka accesible en `http://localhost:8761` desde el navegador
- ✓ El servidor no aparece como instancia registrada de si mismo
- ✓ Al levantar cualquier microservicio cliente aparece en el dashboard como UP
- ✓ No hay errores de conexion rechazada en los logs

Crear docker-compose base con PostgreSQL y Kafka

DESCRIPCION

Archivo `docker-compose.yml` en la raiz del repositorio que levanta toda la infraestructura de datos: PostgreSQL con multiples bases de datos y Apache Kafka con Zookeeper. Punto de partida para cualquier desarrollador que clone el repo.

PASOS

- Servicio postgres: imagen `postgres:15-alpine`, exponer puerto 5432
- Crear las 5 bases de datos via script `init-db.sql`: `db_hitbox_auth`, `db_hitbox_main`, `db_hitbox_stats_maps`, `db_hitbox_stats_weapons`, `db_hitbox_stats_agents`
- Servicio zookeeper: imagen `confluentinc/cp-zookeeper:7.5.0`
- Servicio kafka: imagen `confluentinc/cp-kafka:7.5.0`, exponer puerto 9092
- Healthchecks para postgres y kafka
- Credenciales en archivo `.env` (nunca hardcodeadas en el `yml`)
- Agregar `.env` al `.gitignore` y crear `.env.example` en el repo

CRITERIOS DE ACEPTACION

- ✓ `docker-compose up -d` levanta todos los servicios sin errores
- ✓ `psql` puede conectar a cada una de las 5 bases de datos
- ✓ `kafka-topics.sh --list` no retorna error de conexion
- ✓ Los contenedores se reinician automaticamente con `restart: unless-stopped`
- ✓ El archivo `.env.example` existe con las variables necesarias pero sin valores reales

Estructura de repositorio GitHub con convencion de paquetes

DESCRIPCION

Organizar el repositorio de GitHub con la estructura de carpetas correcta y verificar que todos los proyectos usen la convencion de paquetes acordada. Evita inconsistencias dificiles de corregir despues.

PASOS

- Estructura de carpetas raiz: una subcarpeta por microservicio con el nombre exacto del servicio
- Verificar que el paquete base de cada proyecto sea `cl.nexusarena.[nombre-servicio]`
- Crear README.md en la raiz con: descripcion del proyecto, tabla de puertos de los 13 servicios, prerequisites, instrucciones de arranque
- .gitignore raiz: excluir `target/`, `*.class`, `*.jar`, `application-local.yml`, `.env`
- Crear rama develop para trabajo en curso y main solo para versiones estables

CRITERIOS DE ACEPTACION

- ✓ Cada proyecto tiene exactamente el package base `cl.nexusarena.[servicio]`
- ✓ El README tiene la tabla de puertos completa con los 13 servicios
- ✓ El .gitignore impide subir archivos compilados y credenciales
- ✓ La rama develop existe y esta configurada como rama por defecto para PRs

EPICA 2

Seguridad y Gateway

Implementa la capa de autenticacion y el punto de entrada unico al sistema. Sin esta epica ningun endpoint puede consumirse de forma autenticada desde el frontend.

Implementar ms-auth con registro, login y JWT

DESCRIPCION

Microservicio de autenticacion que maneja registro de usuarios, login y generacion de tokens JWT. Es la puerta de entrada a todo el sistema. Se conecta a su propia base de datos db_hitbox_auth.

PASOS

- Dependencias: Spring Security, Spring Web, Spring Data JPA, PostgreSQL Driver, Eureka Discovery Client, Config Client, Lombok, Validation
- Entidad User: id (UUID), username, email, password (BCrypt), enabled, createdAt
- Entidad Role: id, name (ROLE_USER, ROLE_ADMIN) — relacion ManyToMany con User
- POST /auth/register: valida campos, verifica email no duplicado, hashea password, retorna 201
- POST /auth/login: verifica credenciales, genera JWT firmado con HS256, retorna token con expiracion de 24h
- Clase JwtUtil: generateToken(), validateToken(), extractUsername()
- JwtAuthenticationFilter extends OncePerRequestFilter: intercepta cada request y valida el JWT
- SecurityConfig: rutas /auth/** son publicas, todo lo demas requiere autenticacion
- GET /auth/validate: endpoint interno que el gateway usa para verificar tokens
- Package base: cl.nexusarena.msauth — Puerto: 8081

CRITERIOS DE ACEPTACION

- ✓ POST /auth/register con email ya existente retorna 409 Conflict con mensaje descriptivo
- ✓ POST /auth/login con credenciales correctas retorna JSON con campo token y expiresIn
- ✓ El JWT decodificado contiene username y roles en el payload
- ✓ GET /auth/validate con token valido retorna 200, con token invalido retorna 401
- ✓ Request a endpoint protegido sin Authorization retorna 401
- ✓ La password almacenada en DB nunca es texto plano
- ✓ El servicio aparece en Eureka dashboard como MS-AUTH

Implementar api-gateway con ruteo y filtro JWT

DESCRIPCION

Unico punto de entrada al sistema. Todas las requests del cliente pasan primero por aqui. El gateway valida el JWT antes de rutear la request al microservicio destino usando Eureka para descubrir servicios por nombre logico.

PASOS

- Dependencias: Gateway (Spring Cloud Gateway), Eureka Discovery Client
- Habilitar discovery locator: `spring.cloud.gateway.discovery.locator.enabled=true`
- Crear `AuthenticationFilter` que implementa `GlobalFilter`: extrae token del header, llama a `ms-auth GET /auth/validate`, si retorna 401 rechaza la request
- Lista blanca sin filtro JWT: `POST /api/auth/register`, `POST /api/auth/login`
- Rutas explicitas en `application.yml` para todos los microservicios
- Configurar CORS si hay cliente web
- Package base: `cl.nexusarena.apigateway` — Puerto: 8080

CRITERIOS DE ACEPTACION

- ✓ Request con JWT valido llega correctamente al microservicio destino
- ✓ Request sin JWT a ruta protegida retorna 401 directamente desde el gateway
- ✓ Request a `/api/auth/login` sin token funciona correctamente (whitelist)
- ✓ Todos los servicios registrados en Eureka son descubiertos automaticamente
- ✓ El gateway aparece en Eureka dashboard como API-GATEWAY

Tabla de puertos y verificacion en Eureka

DESCRIPCION

Documentar y verificar los puertos de todos los servicios de infraestructura y seguridad ya levantados. El dashboard de Eureka debe mostrar todos los servicios como UP antes de continuar.

PASOS

- Documentar tabla de puertos completa en `README.md` (13 servicios)
- Verificar que `ms-auth` aparece como UP en Eureka
- Verificar que `api-gateway` aparece como UP en Eureka
- Realizar smoke test inicial confirmando que no hay servicios en DOWN

CRITERIOS DE ACEPTACION

- ✓ La tabla de puertos esta en el `README.md` del repositorio
- ✓ `ms-auth` aparece en Eureka dashboard como UP
- ✓ `api-gateway` aparece en Eureka dashboard como UP
- ✓ No hay servicios en estado DOWN en el dashboard al hacer smoke test

Tests de integracion flujo completo de autenticacion

DESCRIPCION

Pruebas de integracion que verifican el flujo completo: desde el registro hasta hacer una request autenticada que pasa por el gateway y llega a un microservicio real.

PASOS

- Crear coleccion Postman con pasos: register -> login -> request autenticada a /api/weapons
- Test automatizado @SpringBootTest para ms-auth usando H2 en memoria
- Verificar que request a /api/weapons sin JWT retorna 401 desde el gateway
- Verificar que el mismo request con JWT valido retorna 200 con lista de armas
- Exportar la coleccion Postman en /docs/postman/ dentro del repositorio

CRITERIOS DE ACEPTACION

- ✓ Los 3 pasos de la coleccion Postman corren sin errores manuales
- ✓ Test unitario de JwtUtil pasa: generacion y validacion de token funcionan
- ✓ Request sin token al gateway retorna exactamente 401 (no 403, no 500)
- ✓ Request con token valido al gateway llega a ms-weapons y retorna datos reales
- ✓ La coleccion Postman esta exportada y commitada en el repositorio

EPICA 3

Microservicios de Negocio

Los cuatro microservicios CRUD que el training-engine necesita para operar. Cada uno tiene su propia base de datos, DTOs, validaciones y GlobalExceptionHandler.

Implementar ms-weapons con CRUD completo

DESCRIPCION

Microservicio de armas con CRUD completo y datos reales de Valorant. El ms-training-engine lo consulta via OpenFeign de forma sincronica antes de iniciar cada sesion. Si GET /weapons/{id} no funciona el motor de entrenamiento no puede arrancar sesiones.

PASOS

- Dependencias: Spring Web, Spring Data JPA, PostgreSQL Driver, Eureka Discovery Client, Config Client, Lombok, Validation
- Entidad Weapon: id (Long), name, category (Enum: RIFLE/PISTOL/SMG/SNIPER/SHOTGUN/HEAVY), damage, fireRate, magazineSize, price, description, createdAt, updatedAt
- WeaponRepository: findByCategory(), findByName(), findByPriceRange() (custom query), existsByName(), existsByNameAndIdNot()
- WeaponService + WeaponServiceImpl: findAll, findById (lanza ResourceNotFoundException si no existe), findByCategory, create, update, delete
- WeaponController: GET /weapons, GET /weapons/{id}, GET /weapons/category/{cat}, POST /weapons, PUT /weapons/{id}, DELETE /weapons/{id}
- WeaponRequestDTO con validaciones @NotBlank @NotNull @Min @Max y WeaponResponseDTO
- data.sql con minimo 10 armas reales: Vandal, Phantom, Operator, Odin, Ares, Spectre, Bulldog, Guardian, Marshal, Sheriff
- GlobalExceptionHandler: ResourceNotFoundException -> 404, DuplicateResourceException -> 409, MethodArgumentNotValidException -> 400, Exception -> 500
- Package base: cl.nexusarena.msweapons — Puerto: 8082

CRITERIOS DE ACEPTACION

- ✓ GET /weapons retorna lista con minimo 10 armas al arrancar el servicio
- ✓ GET /weapons/{id} con ID existente retorna el arma correcta con todos los campos
- ✓ GET /weapons/{id} con ID inexistente retorna exactamente 404 con mensaje de error en JSON
- ✓ POST /weapons con body invalido (name vacio) retorna 400 con detalle del campo
- ✓ POST /weapons con nombre duplicado retorna 409 Conflict
- ✓ El servicio aparece en Eureka como MS-WEAPONS

Implementar ms-agents con personajes y habilidades

DESCRIPCION

Microservicio de agentes de Valorant con sus habilidades. Incluye relacion entre agente y sus 4 habilidades (Q, E, C, Ultimate). El training-engine consulta este servicio para validar que el agente seleccionado existe antes de iniciar sesion.

PASOS

- Dependencias: Spring Web, Spring Data JPA, PostgreSQL Driver, Eureka Discovery Client, Config Client, Lombok, Validation
- Entidad Agent: id (Long), name, role (Enum: DUELIST/CONTROLLER/SENTINEL/INITIATOR), description, imageUrl
- Entidad Ability: id (Long), name, type (Enum: Q/E/C/ULTIMATE), description, agent (ManyToOne)
- AgentController: GET /agents, GET /agents/{id}, POST /agents, PUT /agents/{id}, DELETE /agents/{id}, GET /agents/{id}/abilities
- data.sql: minimo 8 agentes reales con sus 4 habilidades: Jett, Sage, Phoenix, Omen, Brimstone, Cypher, Breach, Killjoy
- AgentResponseDTO debe incluir lista de habilidades anidada
- Package base: cl.nexusarena.msagents — Puerto: 8083

CRITERIOS DE ACEPTACION

- ✓ GET /agents retorna lista con agentes y sus roles correctos
- ✓ GET /agents/{id}/abilities retorna exactamente las 4 habilidades del agente
- ✓ GET /agents/{id} con ID inexistente retorna 404
- ✓ El servicio aparece en Eureka como MS-AGENTS

Implementar ms-maps con catalogo de escenarios

DESCRIPCION

Microservicio de mapas de Valorant. El training-engine lo consulta via Feign para validar que el mapa seleccionado existe. GET /maps/{id} es el endpoint mas critico.

PASOS

- Dependencias: Spring Web, Spring Data JPA, PostgreSQL Driver, Eureka Discovery Client, Config Client, Lombok, Validation
- Entidad Map: id (Long), name, type (Enum: COMPETITIVE/UNRATED/CUSTOM), difficulty (Enum: EASY/MEDIUM/HARD), description, totalSites
- MapController: GET /maps, GET /maps/{id}, POST /maps, PUT /maps/{id}, DELETE /maps/{id}
- data.sql: mapas reales de Valorant: Ascent, Bind, Haven, Split, Fracture, Pearl, Lotus, Sunset
- Package base: cl.nexusarena.msmaps — Puerto: 8084

CRITERIOS DE ACEPTACION

- ✓ GET /maps retorna minimo 7 mapas al arrancar
- ✓ GET /maps/{id} con ID inexistente retorna 404 — CRITICO para el Feign del training-engine
- ✓ El servicio aparece en Eureka como MS-MAPS

Implementar ms-persistence como orquestador de sesiones

DESCRIPCION

Orquestador central de datos de sesiones de entrenamiento. El training-engine se comunica con este servicio para crear y cerrar sesiones. Aqui se almacena el historial completo de todas las sesiones.

PASOS

- Dependencias: Spring Web, Spring Data JPA, PostgreSQL Driver, Eureka Discovery Client, Config Client, Lombok, Validation
- Entidad TrainingSession: id (UUID), userId, weaponId, agentId, mapId, startTime, endTime, status (Enum: ACTIVE/COMPLETED/ABANDONED)
- Entidad SessionResult: id (Long), session (OneToOne), kills, deaths, accuracy (Double), roundsPlayed, headshots
- POST /sessions: crea sesion con status ACTIVE, retorna el ID de la sesion
- PUT /sessions/{id}/complete: recibe SessionResult en body, cambia status a COMPLETED, persiste resultado
- PUT /sessions/{id}/abandon: cambia status a ABANDONED
- GET /sessions/user/{userId}: historial de sesiones ordenado por startTime DESC
- GET /sessions/{id}: detalle completo de una sesion con su resultado
- Package base: cl.nexusarena.ms persistence — Puerto: 8085

CRITERIOS DE ACEPTACION

- ✓ POST /sessions crea una sesion con status exactamente ACTIVE
- ✓ PUT /sessions/{id}/complete cambia status a COMPLETED y persiste el resultado en SessionResult
- ✓ GET /sessions/user/{userId} retorna el historial ordenado por fecha mas reciente primero
- ✓ Una sesion COMPLETED no puede volver a completarse (retorna 409)
- ✓ El servicio aparece en Eureka como MS-PERSISTENCE

GlobalExceptionHandler y patron DTO en todos los servicios de negocio

DESCRIPCION

Implementar el manejo global de errores y el patron DTO de forma consistente en ms-weapons, ms-agents, ms-maps y ms-persistence. Todos deben responder con el mismo formato de error JSON.

PASOS

- Crear clase ApiError: timestamp (LocalDateTime), status (int), error (String), message (String), path (String)
- Crear GlobalExceptionHandler con @RestControllerAdvice en cada servicio
- Handlers: ResourceNotFoundException -> 404, MethodArgumentNotValidException -> 400 con detalle de campo, Exception -> 500
- Agregar Bean Validation en todos los DTOs de request: @NotNull, @NotBlank, @Min, @Max, @Email donde corresponda
- Cada servicio debe tener: [Entidad]RequestDTO para entrada y [Entidad]ResponseDTO para salida
- Usar @Valid en los controllers para activar la validacion automatica

CRITERIOS DE ACEPTACION

- ✓ Todos los errores tienen formato JSON consistente con timestamp, status, message y path
- ✓ Los stack traces nunca se exponen en las respuestas de error
- ✓ Una request con campo requerido vacio retorna 400 con el nombre del campo que fallo
- ✓ Las entidades JPA no tienen ninguna anotacion de Jackson directamente
- ✓ El formato de error es identico en los 4 servicios de esta fase

Tests unitarios con Mockito para servicios de negocio

DESCRIPCION

Pruebas unitarias con Mockito para la capa de servicio de los microservicios de negocio. Verificar la logica de negocio de forma aislada sin necesidad de levantar la base de datos real.

PASOS

- WeaponServiceTest: mock de WeaponRepository, test de findById con resultado existente, test de findById con ID inexistente (debe lanzar ResourceNotFoundException), test de create con datos validos
- AgentServiceTest: test de findById, test de findAllByRole
- MapServiceTest: test de findById, test de findAll
- SessionServiceTest en ms-persistence: test de create, test de complete, test de complete en sesion ya completada
- Cobertura minima del 60% en la capa service (medible con mvn test jacoco:report)
- Configurar maven-surefire-plugin para que los tests corran con mvn test

CRITERIOS DE ACEPTACION

- ✓ mvn test pasa sin errores en los 4 servicios
- ✓ Los tests no requieren base de datos real (usan mocks de repositorio)
- ✓ El caso de error (ID inexistente -> ResourceNotFoundException) esta cubierto en cada servicio
- ✓ El reporte de Jacoco muestra minimo 60% de cobertura en los paquetes service

Motor de Entrenamiento y Kafka

El nucleo del sistema. Integra comunicacion sincronica (OpenFeign) y asincronica (Kafka) en un mismo flujo. Define el contrato SessionCompletedEvent que conecta el motor con los servicios de estadisticas.

HIT-15

CRITICA

Definir SessionCompletedEvent — contrato del sistema Kafka

DESCRIPCION

DTO SessionCompletedEvent que el training-engine publica en Kafka y los 3 microservicios de estadisticas consumen. Este es el contrato mas critico del sistema — si se define mal todos los consumers deben reescribirse.

PASOS

- Campos: sessionId (UUID), userId (Long), weaponId (Long), agentId (Long), mapId (Long), kills (Integer), deaths (Integer), accuracy (Double), headshots (Integer), roundsPlayed (Integer), durationSeconds (Long), completedAt (LocalDateTime)
- Serializacion con Jackson (no Avro — simplicidad academica)
- Copiar la clase a cada consumer o crear modulo compartido — documentar la decision tomada
- Documentar el JSON de ejemplo en el README con todos los campos y tipos
- Todo el equipo debe revisar y aprobar la estructura antes de continuar

CRITERIOS DE ACEPTACION

- ✓ La clase serializa a JSON y deserializa de JSON sin perdida de ningun campo
- ✓ El JSON de ejemplo esta documentado en el README con todos los campos y tipos
- ✓ Todo el equipo reviso y aprobo la estructura antes de continuar
- ✓ La clase existe en training-engine y en los 3 servicios de stats con estructura identica

Implementar clientes OpenFeign en ms-training-engine

DESCRIPCION

Cientes OpenFeign dentro del training-engine para validar en tiempo real que el arma, mapa y agente seleccionados existen antes de iniciar una sesion. Comunicacion sincronica y bloqueante.

PASOS

- Agregar dependencia OpenFeign (spring-cloud-starter-openfeign)
- Agregar @EnableFeignClients en la clase main del training-engine
- WeaponClient: @FeignClient(name='ms-weapons') con metodo getWeaponById(@PathVariable Long id) -> GET /weapons/{id}
- MapClient: @FeignClient(name='ms-maps') con metodo getMapById(@PathVariable Long id) -> GET /maps/{id}
- AgentClient: @FeignClient(name='ms-agents') con metodo getAgentById(@PathVariable Long id) -> GET /agents/{id}
- PersistenceClient: @FeignClient(name='ms-persistence') para crear y completar sesiones
- @FallbackFactory en cada client: si el servicio no responde lanzar excepcion controlada (no NullPointerException)
- Los clients resuelven el servicio por nombre logico via Eureka — sin URLs hardcodedas

CRITERIOS DE ACEPTACION

- ✓ Feign resuelve ms-weapons por nombre logico via Eureka (verificar en logs que no usa IP hardcodeda)
- ✓ Si ms-weapons esta caido el fallback retorna una excepcion controlada con mensaje descriptivo (no 500)
- ✓ WeaponClient.getWeaponById(999L) cuando el arma no existe propaga el 404 correctamente
- ✓ Los 4 clients estan correctamente inyectados en el TrainingService

Logica de sesion y Kafka producer en ms-training-engine

DESCRIPCION

Logica central del motor de entrenamiento: iniciar sesiones, validar recursos via Feign, completar sesiones y publicar el evento de finalizacion en Kafka. El servicio mas complejo del sistema.

PASOS

- Dependencias: Spring Web, JPA, PostgreSQL, Kafka, Eureka Client, Config Client, OpenFeign, Lombok
- POST /training/session/start: validar weaponId, mapId, agentId via Feign — si 404 retornar 404 al cliente; luego llamar PersistenceClient para crear la sesion; retornar el sessionId
- PUT /training/session/{id}/complete: cerrar sesion en ms-persistence; construir SessionCompletedEvent; publicar en Kafka; retornar 200
- KafkaProducerConfig: JsonSerializer para serializar el evento a JSON
- Si Kafka no esta disponible al publicar: loguear el error y retornar 200 igual (la sesion ya fue persistida — no fallar al usuario)
- Package base: cl.nexusarena.mstrainingengine — Puerto: 8086

CRITERIOS DE ACEPTACION

- ✓ POST /training/session/start con weaponId inexistente retorna exactamente 404 con mensaje descriptivo
- ✓ PUT /training/session/{id}/complete publica el evento en Kafka (verificar con kafka-console-consumer)
- ✓ El evento publicado contiene todos los campos del SessionCompletedEvent con valores correctos
- ✓ Si Kafka esta caido la sesion igual se guarda en ms-persistence y el endpoint retorna 200 (no 500)
- ✓ El servicio aparece en Eureka como MS-TRAINING-ENGINE

Configurar topic SESSION_COMPLETED_TOPIC con 3 particiones

DESCRIPCION

Crear y configurar el topic de Kafka que conecta el training-engine con los 3 microservicios de estadisticas. La configuracion de group.id es critica para garantizar el fan-out paralelo.

PASOS

- @Bean NewTopic en la configuracion del training-engine: nombre SESSION_COMPLETED_TOPIC, 3 particiones, replication-factor 1
- Configurar group.id distintos: stats-maps-group (ms-stats-maps), stats-weapons-group (ms-stats-weapons), stats-agents-group (ms-stats-agents)
- Verificar con kafka-topics.sh --describe que el topic tiene 3 particiones
- Documentar en README la razon de los 3 group.id distintos y el antipatron del mismo group.id

CRITERIOS DE ACEPTACION

- ✓ kafka-topics.sh --describe muestra 3 particiones para SESSION_COMPLETED_TOPIC
- ✓ Cada servicio de stats tiene configurado su propio group.id unico en application.yml
- ✓ Al publicar 1 evento los 3 servicios de stats lo reciben (fan-out confirmado en logs)
- ✓ El antipatron (mismo group.id) esta documentado en el README

Test end-to-end del flujo completo de sesion

DESCRIPCION

Ejecutar y documentar el flujo completo de una sesion de entrenamiento de extremo a extremo para verificar que toda la Epica 4 funciona antes de pasar a analytics.

PASOS

- Levantar todos los servicios hasta ms-training-engine inclusive
- Flujo Postman: register -> login -> POST /training/session/start -> PUT /training/session/{id}/complete
- Verificar en kafka-console-consumer que el evento se publico con los datos correctos
- Verificar en ms-persistence: GET /sessions/user/1 muestra la sesion con status COMPLETED
- Commitear la coleccion Postman actualizada en /docs/postman/

CRITERIOS DE ACEPTACION

- ✓ Los 4 pasos del flujo Postman corren sin errores
- ✓ El evento en Kafka contiene los datos exactos enviados en el paso de complete
- ✓ La sesion en ms-persistence tiene status COMPLETED con el resultado persistido
- ✓ Los logs del training-engine muestran que el evento fue enviado a Kafka exitosamente

EPICA 5

Analytics y Estadisticas

Los cuatro microservicios que consumen eventos de Kafka y exponen los datos analiticos que el dashboard frontend ya consume. Cada uno tiene su propia base de datos independiente.

Implementar ms-stats-maps con consumer Kafka

DESCRIPCION

Microservicio de estadísticas de mapas que consume eventos de Kafka y calcula cual es el mapa donde los usuarios tienen mejor rendimiento. Tiene su propia base de datos db_hitbox_stats_maps.

PASOS

- Dependencias: Spring Web, Spring Data JPA, PostgreSQL Driver, Spring Kafka, Eureka Discovery Client, Config Client, Lombok
- DB propia: db_hitbox_stats_maps
- Entidad MapStat: id (Long), mapId, mapName, totalSessions, avgKills (Double), avgDeaths (Double), avgAccuracy (Double), bestAccuracy (Double), lastUpdated
- @KafkaListener(topics='SESSION_COMPLETED_TOPIC', groupId='stats-maps-group')
- Logica running average: $\text{newAvg} = ((\text{oldAvg} * (n-1)) + \text{newValue}) / n$ donde $n = \text{totalSessions}$
- GET /stats/maps: lista todos los stats ordenados por avgAccuracy DESC
- GET /stats/maps/best: retorna el mapa con mayor avgAccuracy
- GET /stats/maps/{mapId}: estadísticas de un mapa específico
- Package base: cl.nexusarena.msstatmaps — Puerto: 8087

CRITERIOS DE ACEPTACION

- ✓ Al publicar un evento en Kafka ms-stats-maps lo procesa en menos de 3 segundos
- ✓ GET /stats/maps/best retorna el mapa con mayor avgAccuracy de la base de datos
- ✓ GET /stats/maps/{mapId} con mapId sin sesiones retorna 404
- ✓ El servicio tiene su propia DB db_hitbox_stats_maps
- ✓ Al recibir múltiples eventos del mismo mapa el promedio se recalcula correctamente

Implementar ms-stats-weapons con consumer Kafka

DESCRIPCION

Microservicio de estadísticas de armas. Consume los mismos eventos que ms-stats-maps pero con su propio group.id. Calcula cual es la mejor arma por categoría basándose en kills promedio y accuracy.

PASOS

- DB propia: db_hitbox_stats_weapons
- Entidad WeaponStat: id (Long), weaponId, weaponName, category, totalSessions, avgKills (Double), avgAccuracy (Double), avgHeadshots (Double), lastUpdated
- @KafkaListener(topics='SESSION_COMPLETED_TOPIC', groupId='stats-weapons-group')
- Misma lógica de running average que ms-stats-maps
- GET /stats/weapons: todos los stats ordenados por avgKills DESC
- GET /stats/weapons/best: arma con mayor avgKills global
- GET /stats/weapons/best?category=Rifle: mejor arma del tipo Rifle
- GET /stats/weapons/{weaponId}: stats de un arma específica
- Package base: cl.nexusarena.msstatsweapons — Puerto: 8088

CRITERIOS DE ACEPTACION

- ✓ Recibe el mismo evento que ms-stats-maps en paralelo — ambos logs muestran el mismo sessionId
- ✓ GET /stats/weapons/best?category=Sniper retorna solo el mejor sniper
- ✓ GET /stats/weapons/best?category=XYZ con categoría inexistente retorna 404
- ✓ La DB db_hitbox_stats_weapons es independiente y tiene sus propias tablas

Implementar ms-stats-agents con consumer Kafka

DESCRIPCION

Microservicio de estadísticas de agentes. Calcula el KD ratio por agente y permite filtrar por rol. Consume del mismo topic con su propio group.id independiente.

PASOS

- DB propia: db_hitbox_stats_agents
- Entidad AgentStat: id (Long), agentId, agentName, role, totalSessions, avgKills (Double), avgDeaths (Double), kdRatio (Double), avgAccuracy (Double), lastUpdated
- @KafkaListener(topics='SESSION_COMPLETED_TOPIC', groupId='stats-agents-group')
- Cálculo de KD ratio: $kdRatio = avgKills / Math.max(avgDeaths, 1.0)$ — evitar división por cero
- GET /stats/agents: todos los agentes ordenados por kdRatio DESC
- GET /stats/agents/best: agente con mayor kdRatio global
- GET /stats/agents/best?role=Duelist: mejor Duelist
- GET /stats/agents/{agentId}: stats de un agente específico
- Package base: cl.nexusarena.msstatsagents — Puerto: 8089

CRITERIOS DE ACEPTACION

- ✓ El kdRatio se calcula correctamente: 10 kills / 2 deaths = 5.0
- ✓ El kdRatio nunca es NaN ni Infinity aunque deaths sea 0 (usar Math.max con 1.0)
- ✓ GET /stats/agents/best?role=Controller retorna solo el mejor Controller
- ✓ Los 3 servicios (maps, weapons, agents) reciben el mismo evento en paralelo

Implementar ms-analytics-core con agregacion y cache

DESCRIPCION

Microservicio de analytics central que agrega los datos de los 3 servicios de estadísticas y los expone en un solo endpoint. Usa OpenFeign para consultar los stats services e implementa cache con Caffeine.

PASOS

- Dependencias: Spring Web, Spring Data JPA, PostgreSQL Driver, OpenFeign, Spring Cache, Caffeine, Eureka Discovery Client, Config Client, Lombok
- StatsMapClient, StatsWeaponClient, StatsAgentClient — Feign clients a cada servicio de stats
- PersistenceClient — para consultar historial de sesiones de un usuario
- GET /analytics/summary: agrega mejor mapa, mejor arma, mejor agente en un solo JSON
- GET /analytics/user/{userId}: historial de sesiones del usuario via ms-persistence + estadísticas personales
- GET /analytics/top: top 3 de cada categoría (mapas, armas, agentes)
- @Cacheable('analytics-summary') en el método de summary — cache de 5 minutos con Caffeine
- Si un stats service está caído: retornar datos parciales con campo available: false (no retornar 500)
- Package base: cl.nexusarena.msanalyticscore — Puerto: 8090

CRITERIOS DE ACEPTACION

- ✓ GET /analytics/summary retorna datos de los 3 servicios de stats en un solo JSON
- ✓ GET /analytics/top retorna top 3 de mapas, armas y agentes con sus stats
- ✓ Si ms-stats-maps está caído el endpoint retorna los datos de weapons y agents igualmente
- ✓ La segunda llamada a GET /analytics/summary en menos de 5 minutos viene del cache (verificar en logs)
- ✓ El servicio aparece en Eureka como MS-ANALYTICS-CORE

Verificar fan-out paralelo de Kafka entre los 3 consumers

DESCRIPCION

Prueba formal que verifica que los 3 microservicios de estadísticas consumen el mismo evento en paralelo. Confirma que la configuración de group.id es correcta.

PASOS

- Publicar 1 evento de prueba manualmente con kafka-console-producer
- Tener los logs de los 3 servicios abiertos simultáneamente (3 terminales)
- Verificar que el mismo sessionId aparece en los logs de los 3 servicios
- Consultar las 3 bases de datos y verificar que cada una actualizó su registro
- Documentar el resultado con screenshot de las 3 terminales como evidencia
- Intentar el mismo test con los 3 servicios en el mismo group.id y documentar el antipatrn

CRITERIOS DE ACEPTACION

- ✓ El mismo sessionId aparece en los logs de ms-stats-maps, ms-stats-weapons Y ms-stats-agents
- ✓ Cada DB tiene el registro actualizado después del evento de prueba
- ✓ El screenshot como evidencia está comprometado en /docs/tests/
- ✓ El antipatrn (mismo group.id) está documentado con su resultado en el README

Test de integracion completo con los 13 servicios

DESCRIPCION

Ejecutar el flujo completo del sistema con los 13 microservicios levantados simultaneamente. Test final que confirma que todo funciona de extremo a extremo y que esta listo para la entrega academica.

PASOS

- Levantar todos los servicios en orden correcto: docker-compose -> config-server -> eureka -> api-gateway -> ms-auth -> ms-weapons -> ms-agents -> ms-maps -> ms-persistence -> ms-training-engine -> ms-stats-maps -> ms-stats-weapons -> ms-stats-agents -> ms-analytics-core
- Verificar que los 13 servicios aparecen en Eureka como UP
- Flujo completo: registrar usuario, login, iniciar sesion de entrenamiento, completar sesion, esperar 3 segundos, GET /analytics/top, GET /analytics/user/{userId}
- Documentar tiempo total de prueba y cualquier error encontrado
- README debe tener seccion 'Como correr el sistema completo desde cero'

CRITERIOS DE ACEPTACION

- ✓ Los 13 servicios aparecen como UP en Eureka al mismo tiempo
- ✓ El flujo completo de 7 pasos termina sin errores
- ✓ GET /analytics/top refleja los datos de la sesion completada
- ✓ El historial del usuario muestra la sesion con status COMPLETED y los stats correctos
- ✓ El README tiene instrucciones paso a paso para levantar el sistema desde cero

Documentacion tecnica final del proyecto

DESCRIPCION

Completar toda la documentacion tecnica del proyecto para la entrega academica. Incluye README actualizado, coleccion Postman exportada y diagrama de arquitectura.

PASOS

- README.md raiz actualizado: descripcion, tabla de puertos, requisitos, instrucciones paso a paso, ejemplos de requests con curl o Postman
- Diagrama de arquitectura como imagen PNG en /docs/architecture/ mostrando los 13 servicios con sus relaciones
- Coleccion Postman completa exportada en /docs/postman/hitboxking.postman_collection.json
- Seccion en README: 'Estructura del evento SessionCompletedEvent' con JSON de ejemplo
- Seccion en README: 'Explicacion de la arquitectura' describiendo por que Kafka para fan-out y Feign para validacion sincronica
- Configurar SpringDoc/Swagger en cada microservicio de negocio

CRITERIOS DE ACEPTACION

- ✓ Un desarrollador nuevo puede levantar el sistema desde cero siguiendo solo el README
- ✓ La coleccion Postman tiene los requests listos para ejecutar con variables de entorno para el JWT
- ✓ El diagrama de arquitectura muestra los 13 servicios con sus relaciones
- ✓ El README tiene la tabla de puertos completa y actualizada
- ✓ Swagger accesible en cada microservicio de negocio en /swagger-ui.html